JAVA EXCEPTION HANDLING

1. WHAT IS EXCEPTION ?

An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.
In Java, exceptions are objects that represent errors or unexpected situations.

Examples of Exceptions

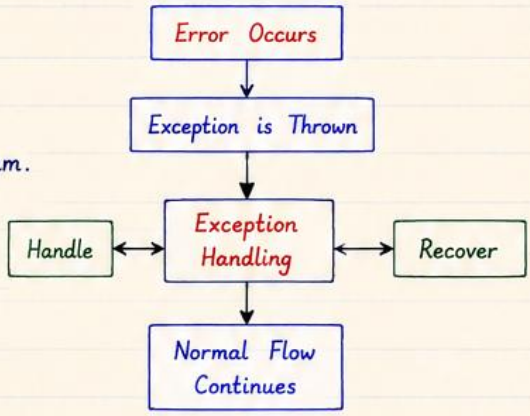
- ★ Division by zero
- ★ File not found
- ★ Null pointer access
- ★ Array index out of range
- ★ Invalid user input

Exception Handling is the mechanism to handle runtime errors so that the normal flow of the program can be maintained.

Note :
Exceptions are checked at runtime (not at compile time).

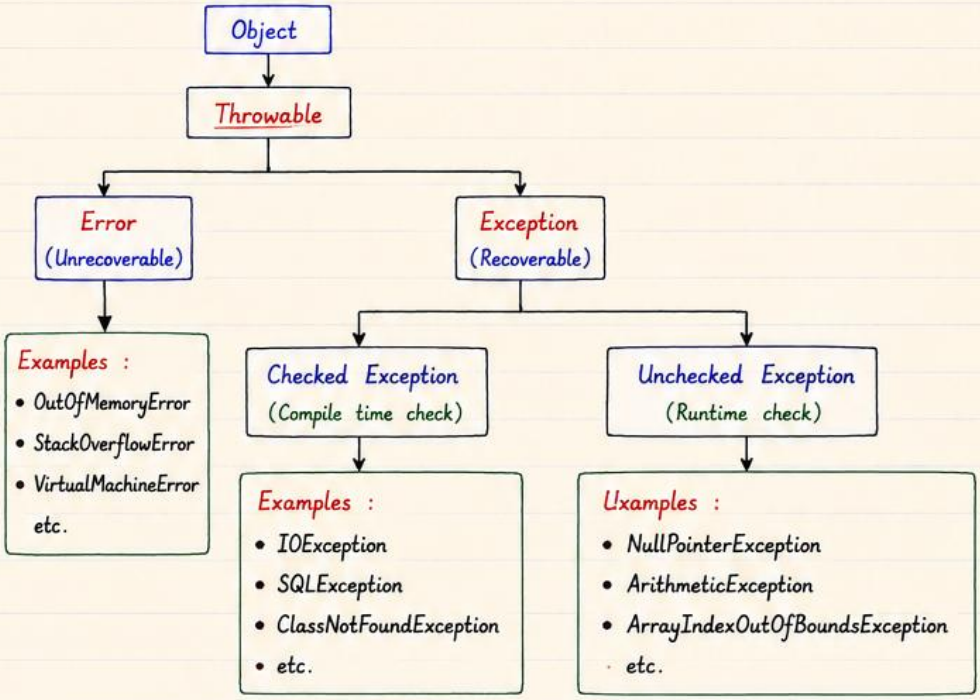
2. WHY EXCEPTION HANDLING ?

- ★ Maintains normal flow of the application.
- ★ Avoids abnormal termination of the program.
- ★ Helps to find and fix bugs easily.
- ★ Provides better user experience.
- ★ Improves code reliability and robustness.



3. EXCEPTION HIERARCHY IN JAVA

In Java, all exceptions are derived from the Throwable class.



Key Points :

- ★ Error → Serious problems (not handled).
- ★ Exception → Can be handled by our code.

JAVA EXCEPTION HANDLING

★ Types of Exceptions

In Java, exceptions are mainly of two types.

1. Checked Exceptions – Checked at compile time.
2. Unchecked Exceptions – Checked at runtime.

Checked Exceptions	Unchecked Exceptions
• Checked at compile time • Must be handled or declared using throws • Subclasses of Exception • Example: IOException FileNotFoundException	• Checked at runtime • Not required to handle • Subclasses of RuntimeException • Example: NullPointerException ArithmeticException

★ Common Built-in Exceptions

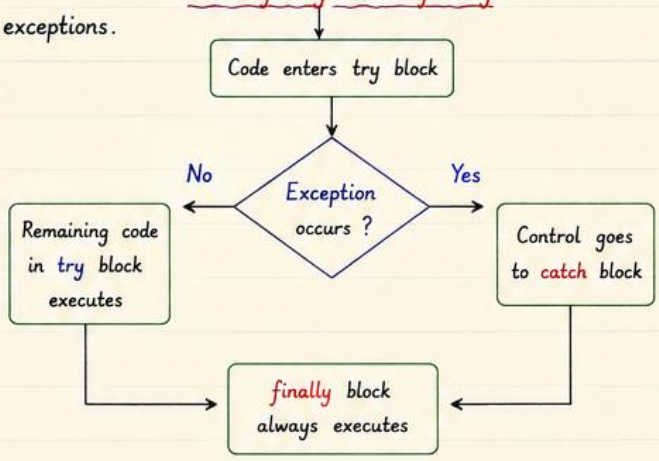
Exception	Description
ArithmeticException	Occurs when an exceptional arithmetic condition occurs (e.g., divide by zero)
NullPointerException	Occurs when an object reference is null
ArrayIndexOutOfBoundsException	Occurs when accessing invalid index in an array
NumberFormatException	Occurs when converting invalid String to number
IOException	Occurs when an input-output operation fails
ClassNotFoundException	Occurs when a class is not found

★ try, catch, finally

We use try, catch and finally block to handle exceptions.

```
try {
   // code that may cause exception
   int a = 10, b = 0;
   int c = a / b;   // may throw ArithmeticException
   System.out.println(c);
}
catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
finally {
    System.out.println("Finally block executed");
}
```

Flow of try-catch-finally



★ throw and throws

throw

- Used to explicitly throw an exception.
- Used inside the method.
- Syntax:

```
throw exceptionObject;
```

Example:

```
if(age < 18) {
   throw new ArithmeticException("Age must be 18 or above");
}
```

throws

- Used to declare an exception.
- Used in method signature.
- Indicates that the method may throw an exception.
- Syntax:

```
returnType methodName() throws ExceptionType;
```

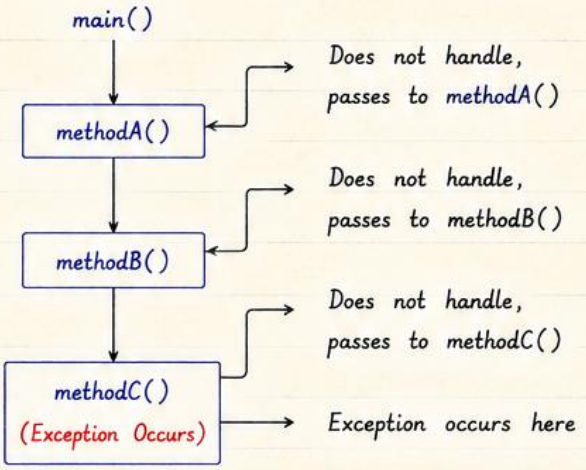
Example:

```
int divide(int a, int b) throws ArithmeticException {
   if(b == 0)
      throw new ArithmeticException("Cannot divide by zero");
   return a / b;
}
```

JAVA EXCEPTION HANDLING

★ Exception Propagation

If an exception is not handled in a method, it is passed to the calling method. This continues up the call stack until the exception is handled.



Note :
If no method handles the exception, the program terminates abnormally and prints exception message.

★ Custom Exception

We can create our own exceptions by extending Exception class.

```
class InvalidAgeException extends Exception {
   InvalidAgeException(String msg) {
      super(msg);                          // user defined
   }                                       // exception
}
```

★ Multiple catch Blocks

A try block can have multiple catch blocks to handle different types of exceptions.

```
try {
   int a = 10, b = 0;
   int c = a / b;        // may throw ArithmeticException
   int arr[] = new int[2];
   arr[5] = 10;          // may throw ArrayIndexOutOfBoundsException
}
catch (ArithmeticException e) {
   System.out.println("Arithmetic Exception");
}
catch (ArrayIndexOutOfBoundsException e) {
   System.out.println("Array Index Exception");
}
catch (Exception e) {
   System.out.println("Some other Exception");
}
```

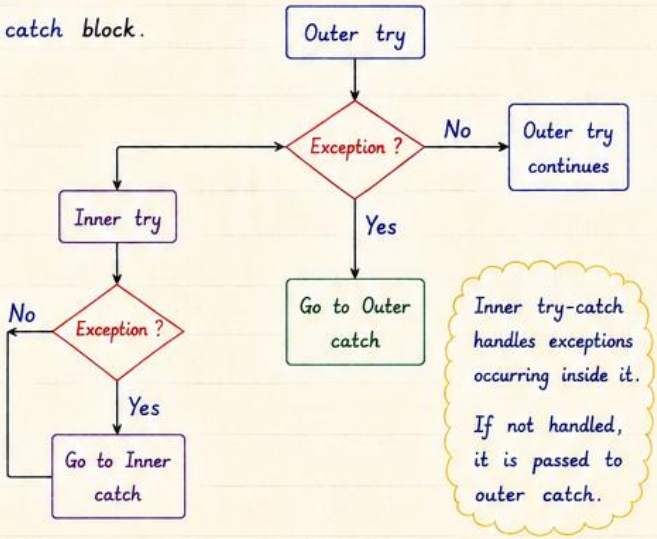
Order of catch blocks

- Specific exceptions must be written before general exceptions.
- General catch (Exception e) should be last.
- If general catch is written first, compiler will give error.

★ Nested try Block

A try block can be placed inside another try or catch block.

```
try {
   int a[] = new int[2];
   a[5] = 30;        // may throw ArrayIndexOutOfBoundsException
   try {
      int x = 10 / 0;     // may throw ArithmeticException
   }
   catch (ArithmeticException e) {
      System.out.println("Inner try-catch");
   }
}
catch (ArrayIndexOutOfBoundsException e) {
   System.out.println("Outer try-catch");
}
```



Inner try-catch handles exceptions occurring inside it.
If not handled, it is passed to outer catch.

MULTITHREADING IN JAVA

B. MULTITHREADING

★ What is Thread ?

A thread is the smallest unit of execution in a program.

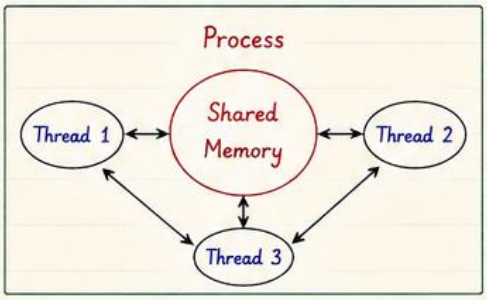
A program can have multiple threads running concurrently to perform tasks.

Why Multithreading ?

- Improves performance.
- Better resource utilization.
- Enables concurrent execution.
- Keeps application responsive.
- Essential for modern applications.

★ Benefits of Multithreading

- Responsiveness – UI remains responsive.
- Performance – Tasks run in parallel.
- Resource sharing – Threads share memory.
- Economy – Less memory & context switching.
- Scalability – Handles more tasks simultaneously.



★ Creating a Thread

There are two ways to create a thread in Java.

1. By extending Thread class

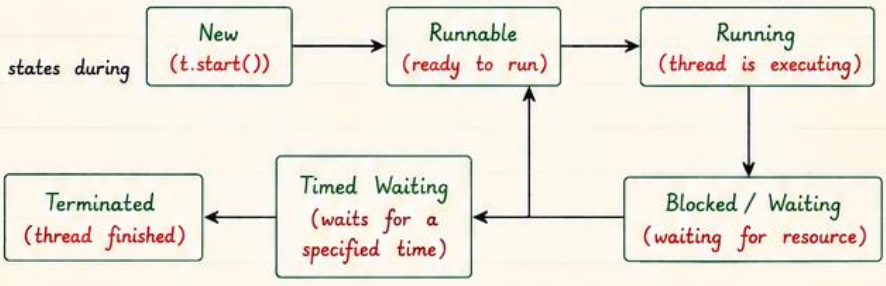
```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}
public class Test {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();      // starts the thread
    }
}
```

2. By implementing Runnable interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
}
public class Test {
    public static void main(String[] args) {
        MyRunnable r = new MyRunnable();
        Thread t = new Thread(r);
        t.start();      // starts the thread
    }
}
```

★ Thread Life Cycle

A thread goes through different states during its life cycle.



★ start() vs run()

Feature	start()	run()
Who starts	JVM starts a new thread	Called like a normal method
New thread created	Yes	No
Executes in parallel	Yes	No
Calls	start() internally calls run()	Directly executes run()
Example	t.start();	t.run();

MULTITHREADING IN JAVA

★ Thread Methods

Method	Description
start()	Starts the execution of thread. It internally calls run().
run()	Contains the code to be executed by the thread.
sleep(long millis)	Pauses the current thread for specified time.
join()	Waits for this thread to die.
yield()	Causes the currently executing thread to pause & allow other threads of same priority to execute.
isAlive()	Tests if the thread is alive.

★ Example : Using sleep() and join()

```
class SleepJoinExample extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Thread running : " + i);
            try {
                Thread.sleep(500);    // sleep for 0.5 sec
            } catch(InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
public class Test {
    public static void main(String[] args) throws Exception {
        SleepJoinExample t = new SleepJoinExample();
        t.start();
        t.join();    // main thread waits for t to finish
        System.out.println("Thread has finished");
    }
}
```

Output

```
Thread running : 1
Thread running : 2
Thread running : 3
Thread running : 4
Thread running : 5
Thread has finished
```

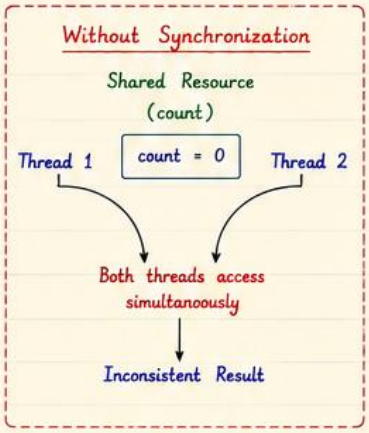
Note :
join() makes the main thread wait until the child thread completes.

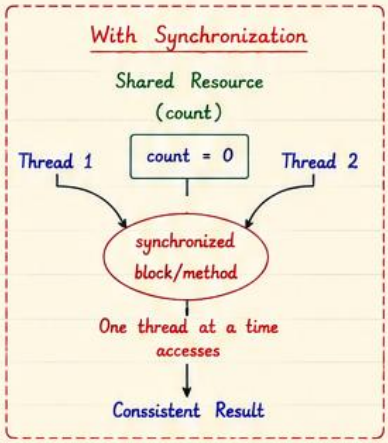
★ Synchronization (synchronized)

Synchronization is used to control access to shared resources to avoid inconsistent data.

```
class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```





★ Daemon Thread

A daemon thread is a low priority thread that runs in background to perform tasks like garbage collection.

```
Thread t = new Thread(new MyTask());
t.setDaemon(true);  // set as daemon thread
t.start();
```

MULTITHREADING IN JAVA

★ Thread Priorities

A thread priority helps to determine the order in which threads are scheduled for execution.

Java supports 3 priority levels.

Priority Constant	Value	Description
Thread.MIN_PRIORITY	1	Lowest priority
Thread.NORM_PRIORITY	5	Normal priority (default)
Thread.MAX_PRIORITY	10	Highest priority

```
Thread t1 = new Thread();
Thread t2 = new Thread();

t1.setPriority(Thread.MIN_PRIORITY);     // 1
t2.setPriority(Thread.MAX_PRIORITY);     // 10
System.out.println("t1 priority: " + t1.getPriority());
System.out.println("t2 priority: " + t2.getPriority());
```

★ Inter Thread Communication

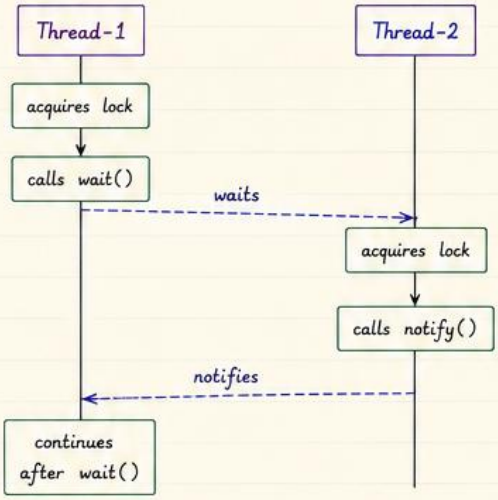
Threads can communicate with each other using wait(), notify() and notifyAll() methods of Object class.

```
class Communication {
  public static void main(String[] args) {
    final Object lock = new Object();

    Thread t1 = new Thread(() -> {
      synchronized (lock) {
        System.out.println("Thread-1 : waiting...");
        try { lock.wait(); } catch (InterruptedException e) {}
        System.out.println("Thread-1 : notified!");
      }
    });
    Thread t2 = new Thread(() -> {
      synchronized (lock) {
        try { Thread.sleep(1000); } catch (InterruptedException e) {}
        System.out.println("Thread-2 : notifying...");
        lock.notify();
      }
    });
    t1.start();
    t2.start();
}
```

Note :
wait(), notify() and notifyAll() must be called from synchronized block or method.

How it works ?

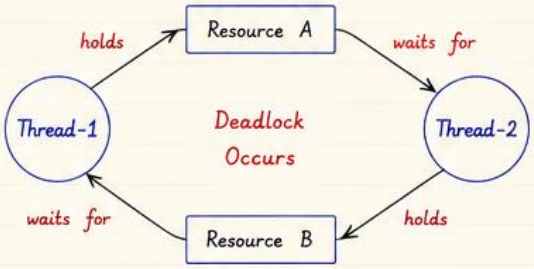


★ Deadlock

Deadlock occurs when two or more threads are blocked forever, each waiting for a resource held by the other.

Example Scenario

Thread-1 holds Resource A and waits for B
Thread-2 holds Resource B and waits for A
Both threads are waiting for each other forever.



★ Important Points

- Always start a thread using start() method, not run().
- Use synchronization to avoid data inconsistency.
- Use proper priority and communication to control threads.
- Avoid deadlock by acquiring locks in fixed order.
- Daemon thread does not prevent JVM from exiting.

```
Thread t = new Thread(new MyTask());
t.setName("Worker-1");
t.setDaemon(true);      // daemon thread
t.setPriority(Thread.NORM_PRIORITY);
t.start();
```

COMMENT PDF

TO GET

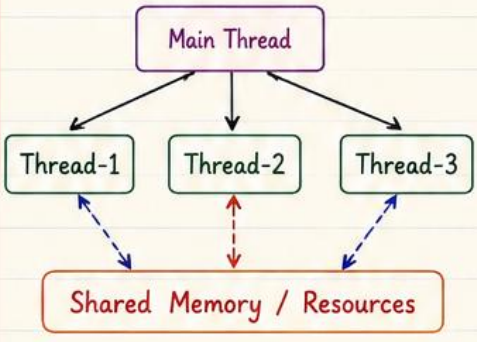
COMPLETE PDF

OF

MULTITHREADING AND EXCEPTION HANDLING

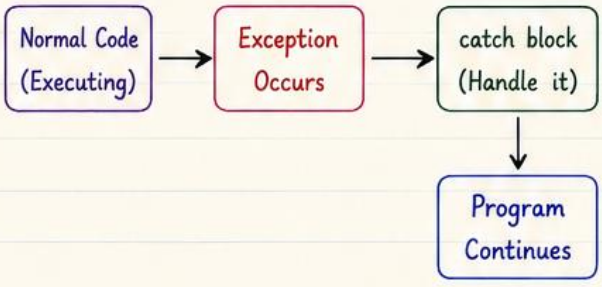
MULTITHREADING

- Multiple threads execute concurrently.
- Threads share the same memory.
- Improves performance and responsiveness.



EXCEPTION HANDLING

- Exception is an event that disrupts normal flow.
- Handled using try, catch, finally, throw, throws.
- Keeps the program stable and error-free.



SAVE | SHARE | FOLLOW

code_with_aman